

Lecture: Sorting algorithms

Week 3. We need a shared vocabulary before the assignment.

A sorting algorithm takes n comparable items and permutes them into non-decreasing order. The interesting questions are: how many comparisons, how much extra memory, and whether equal keys keep their original order (stability).

Insertion sort

Walk left to right. For each item, slide it left until it sits in the already-sorted prefix.

- Best case: already sorted, $\Theta(n)$ comparisons.
- Worst case: reverse sorted, $\Theta(n^2)$.
- In-place. Stable if you insert after equals, not before.

Use it for tiny arrays, or as the base case of a recursive sort once the slice is under ~ 32 elements. A lot of “slow” sorts win on cache at that size.

Mergesort

Split the array in half, sort each half, merge the two sorted runs.

- Recurrence $T(n) = 2 T(n/2) + \Theta(n) \rightarrow \Theta(n \log n)$ always.
- Needs $\Theta(n)$ extra memory for the merge buffer.
- Stable, which is why it is the default when you care about a secondary key.

The merge step is the whole idea: two pointers, copy the smaller head, drain the remainder. If you can write merge correctly you can write mergesort.

Quicksort

Pick a pivot, partition so smaller keys go left and larger go right, recurse.

- Average $\Theta(n \log n)$. Worst case $\Theta(n^2)$ if every pivot is an extreme.
- In-place (ignoring recursion). Not stable in the usual partition.
- Practical fix: median-of-three or random pivot, and fall back to insertion sort on small slices.

Do not claim “quicksort is always faster than mergesort”. On objects with an expensive comparison, or when stability matters, mergesort is the grown-up default.

What the assignment expects

You will visualise insertion sort and mergesort, not quicksort. That is deliberate: the two algorithms make opposite trade-offs (in-place vs guaranteed $n \log n$), and the animation should make that visible.

If a step does not move any bars, it is still a step. Count comparisons, not swaps, when you talk about cost in the write-up.